# Contents


**1st tutorial** **1**

1 Missing Number . . . . . 1
2 Sorted Array Fixed Point . . . . . 1
3 Egg testing . . . . . 1
4 Laser . . . . . 2
5 Submatrix search . . . . . 3
6 Fibonacci sequence . . . . . 3

**2nd tutorial** **4**

1 Comparing functions . . . . . 4
2 Properties of asymptotic notation . . . . 5
3 DFS Runs . . . . . 7
4 Singly Connected Graphs . . . . . 7
5 Longest Path In a DAG . . . . . 8
6 Count Paths In a DAG . . . . . 8

**2nd tutorial — SYGA** **10**

1 Comparing functions . . . . . 10
2 Properties of asymptotic notation . . . . 11
3 DFS Runs — SYGA . . . . . 13
4 Singly Connected Graphs — SYGA . . 13
5 Longest Path In a DAG . . . . . 14
6 Count Paths In a DAG . . . . . 15

**3rd tutorial** **16**

1 Peeling a Graph . . . . . 16
2 Parallel Planning . . . . . 16
3 Computopia . . . . . 17
4 Semi-connectedness. . . . . 17

**4th tutorial** **18**

1 Transitive Reduction . . . . . 18
2 Bipartite Graph Check . . . . . 18
3 Broken Taxi . . . . . 18
4 #Shortest Paths . . . . . 19
5 Pouring Water . . . . . 19
6 Ghost . . . . . 19
7 Guard . . . . . 20

Jamie Fravel
jfravel@iuuk.mff.cuni.cz

# Parameterization

**1. Missing Number** We are given an unsorted array $A$ which contains $n$ pairwise different numbers from set $\{0, \ldots, n\}$. One number from this set is missing in $A$. Describe an efficient algorithm that finds the missing number. Try to describe an algorithm that uses only a constant amount of additional memory.

**Proposition 1.1** (Gauss's sum)**.** $\sum_{i=1}^{n} i = \frac{n(n-1)}{2}$

The missing number $m$ can be derived from Gauss's sum. In particular,

$$m = \frac{n(n-1)}{2} - \sum_{i=1}^{n} A[i].$$

The sum requires $n$ operations but, by doing it in-place, takes only one variable (constant memory).

**2. Sorted Array Fixed Point** We are a given an array $A$ of $n$ integers that are sorted in increasing order. The range of the integers is not bounded in this case, in particular, numbers can be negative. Describe an efficient algorithm which finds an index $i$ such that $A[i] = i$.

**Bonus.** Does your approach need to change if $A$ contains some duplicate entries?

Consider the array $B = \left[A[i] - i\right]_{i=1}^{n}$. What are we looking for in this array?

**Bonus.** Notice that standard binary search fails on $A = [1, 1, 4, 6, 6]$. Then $B = [0, -1, 1, 2, 1]$.
**Proposition 1.2.** *For an index $j$, let $m = A[j]$. If $j < m$, then $i \notin (m, j]$.*

Without duplicate entries, $A$ is strictly increasing and standard binary search works just fine.

**Bonus.** With duplicate entries, we can only prune the entries between $i$ and $A[i]$, not the entire half-array. This ends up giving us a worst case complexity $\mathcal{O}(n)$.

**3. Egg testing** The Empire State Building is a 102-story skyscraper, and we know that if we throw an egg from the $K$-th floor or higher, it will break. Unless the egg breaks, it can be collected and reused. We want to determine $K$, but use as few attempts (throws) as possible. What's the best strategy (minimizing the number of throws) if we have

a) one egg,

b) unlimited eggs,

c) two eggs,

d) **Bonus.** three eggs, or in general $e \in \mathbb{N}$ eggs?

(The eggs can be special, so nothing can be *a priori* assumed about $K$.)

Binary search is efficient because it rules out large chunks of the array at every step. But this can't work with only two eggs... right?

Suppose that we throw the first egg from floors $m$, $2m$, $3m$ etc. How many throws do we use in the worst case?

a) With only one egg, we are forced to perform a linear search. We can throw the egg from each floor in increasing order. This requires $K \leq N$ attempts.

b) With an unlimited number of eggs, we can perform a binary search! Throw from the middle floor, disregard half of the floors and repeat until $k$ is found.

c) With two eggs, we cannot complete a binary search, but the concept is still useful. If the first egg is used to rule out some set of the floors, then the problem reduces to the one egg case on a smaller building.

Binary search searches the midpoint because it never needs to reduce to a linear search. In this two-stage search we may be better served by a small second stage (by skipping a smaller number of floors). Suppose we skip $m$ floors at a time; in the worst case we will need $\frac{102}{m} + m$ throws. The minimum of this function occurs at $m = \sqrt{102}$ giving $2\sqrt{102}$ throws. If we had $n$ floors, then $m$ would need to be $\sqrt{n}$.

d) **Bonus.** With three eggs, we can do add a second skip-stage to the above scheme. After the first egg, we will have the two-egg case on a smaller building! Say we skip $m_1$ then $m_2$ floors with the first then second egg respectively; the worst case is now $\frac{102}{m_1} + \frac{m_1}{m_2} + m_2$ which is minimized at $m_1 = n^{\frac{2}{3}}$ and $m_2 = n^{\frac{1}{3}}$. Notice that this lines up with the two egg case ($m_2 = \sqrt{m_1}$) and uses at most $3n^{\frac{1}{3}}$ throws. Following this formula up to $e$ eggs gives us $\Theta(n^{\frac{1}{e}})$ many throws.

**4. Laser** There is a row of $N$ buildings with $h_1, \ldots, h_n$ floors, and we need to demolish all of them. To that end, you found at home a demolition laser which is capable of firing *vertically* to destroy an arbitrary building or *horizontally* to destroy a given floor in all buildings (i.e., if you choose to destroy floor $L$, then the number of floors decreases by 1 for all buildings whose number of floors is $\geq L$). Develop an algorithm to determine minimum number of firings necessary to eliminate all buildings? (Beware that the maximum number of floors can be much more than $N$.)

Assume that every vertical firing targets the tallest remaining building and that every horizontal firing targets the ground floor. Clearly, there exists an optimal firing strategy which follows these rules. Now, let $u$ and $v$ represent the number of horizontal and vertical firings respectively. Define $f(v)$ to be the number of floors in the $v^{\text{th}}$ tallest building. If $h$ is sorted, then $f(v) = h_{n-v}$.

**Proposition 1.3.** *An optimal firing strategy will minimize $f(v) + v$.*

**Algorithm** `vParamLaser`$(h)$

```
h ← sort(h)
v* ← 0,   f* ← 0
for v ∈ [1 : n] do
    if h_{n−v} + v < f* + v* then
        v* ← v,   f* ← h_{n−v}
    end if
end for
return (f(v), v)
```

**Algorithm** `uParamLaser`$(h)$

```
h ← sort(h)
i ← 1,   g ← 0
u* ← 0,   g* ← 0
for u ∈ [1 : h_n] do
    while h_i ≤ u do
        i ← i + 1,   g ← g + 1
    end while
    if u + g < u* + g* then
        v* ← v,   f* ← h_{n−v}
    end if
end for
return (u, g(u))
```

**Proposition 1.4.** *The algorithm* `vParamLaser` *returns a minimal firing pattern in $\mathcal{O}(n)$ time.*

*Proof.* The algorithm has a sort call (counting sort is linear time) and one loop containing only constant time operations. Said loop will run $n$ times because there are only $n$ buildings to destroy. Thus, $\mathcal{O}(n)$. Knowing that $v = n$ destroys all buildings, we loop over every relevant instance, saving the minimum-known strategy at each step. □

Interestingly, following similar logic while parameterized on $u$ rather than $v$ results in a polynomial algorithm. This is because, unlike $v$, we don't know exactly how many buildings get destroyed as we increase $h$. It could be none. It could be all $n$. We end up spending the extra time computing $g(u)$ even if $h$ is sorted. See `uParamLaser`.

**5. Submatrix search** We are given an integer matrix $A$ with $n$ columns and $m$ rows. Describe an efficient algorithm that finds a maximum submatrix of $A$ consisting only of values 0 (i.e. the submatrix with largest area). Can you achieve complexity $O(nm)$?

Each row $j$ has at most $n$ intervals $(i_1, i_2)$ of zero.

The first idea is to check all top left corners $(i_1, j_1)$ and all bottom right corners $(i_2, j_2)$ and check if a submatrix given by these corners contains all zeros, then pick the largest such submatrix. This would lead to an algorithm of complexity $\Theta(n^3m^3)$.

A better idea is to consider, for every $j = 1, \ldots, m$, the intervals $(i_1, i_2)$ of zeroes on row $j$. There is only at most $n$ of these intervals. Then, we try to extend this interval down to find a maximal $j_2 > j_1$ such that submatrix with top left corner $(i_1, j_1)$ and bottom right corner $(i_2, j_2)$ is full of zeros. This takes only time $O(n^2m^2)$ altogether.

The previous idea can be improved. For every cell of the matrix at coordinates $(i, j)$, we consider number $c(i, j)$ that denotes the number of zeros to the right of $A[i, j]$ before some nonzero value (i.e. max $k$ such that $A[i + t, j] = 0$ for $t = 0, \ldots, k$). Then in the above approach, $j_2$ can be found in linear time. By symmetry, this gives an algorithm of overall complexity $O(nm \min m, n)$.

If we also precompute the number of zeros above, below and to the left of each cell, we can further optimize the above approach to get an $O(nm)$ time algorithm.

**6. Fibonacci sequence** Fibonacci sequence is defined using $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$.

a) What is the value of $F_n$ for a given $n$?

b) Describe an algorithm for computing $F_n$ using only $\Theta(\log_2 n)$ arithmetic operations.

a) Suppose that $F_n$ takes the form $r^n$. What does $r$ need to be? Does this work for $n = 0$ or $n = 1$?

b) Try representing the recursion in the form of matrix multiplication.

a) For $n \geq 0$ we have that

$$F_n = \frac{1}{\sqrt{5}} \cdot \left( \left( \frac{1+\sqrt{5}}{2} \right)^n - \left( \frac{1-\sqrt{5}}{2} \right)^n \right).$$

Asymptotically, $F_n = \Theta(\varphi^n)$ where $\varphi = \frac{1+\sqrt{5}}{2}$ is the Golden ratio.

Unfortunately $\sqrt{5}$ is irrational, so actually computing this value is not possible on any modern computer. Hence part b.

b) The Fibonacci recursion can be thought of a shifting window of three values, $F_n, F_{n-1}, F_{n-2}$. Shifting the window can be interpreted as a linear transformation and thus as a matrix multiplication:

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} F_{n-2} \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} F_{n-1} \\ F_n \end{pmatrix}$$

Since matrix multiplication is an associative operation, we have

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n \cdot \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_{n-1} \\ F_n \end{pmatrix}$$

We can compute exponents (including matrix exponents) in logarithmic time by repeated squaring according to the binary representation of $n$.

# Asymptotics

**1. Comparing functions** Determine the asymptotic relationships between each pair of functions $f$ and $g$ in the following table (i.e. whether $f \in \mathcal{O}(g)$, $f \in \Omega(g)$ or $f = \Theta(g)$).

Lets start with a few different definitions of $\mathcal{O}$, $\Omega$ and $\Theta$.

Intuitively, $f \in \mathcal{O}(g)$ if $f$ grows at *most* as fast as $g$; $f \in \Omega(g)$ if $f$ grows at most as *least* as $g$ and $f \in \Theta(g)$ if $f$ grows *about* as fast as $g$.

Another intuition could be $f \in \mathcal{O}(g)$ if $g(n)$ never becomes irrelevantly small when compared to $f(n)$; $f \in \Omega(g)$ if $f(n)$ never becomes irrelevantly small when compared to $g(n)$; grows at *most* as fast as $g$ and $f \in \theta(g)$ if $f$ and $g$ never become irrelevant compared to each other.

Formally, $\mathcal{O}$, $\Omega$ and $\Theta$ are defined:

- $f \in \mathcal{O}(g)$ if there exist constants $c, m > 0$ such that $f(n) \leq cg(n)$ for all $n > m$.
- $f \in \Omega(g)$ if there exist constants $c, m > 0$ such that $f(n) \geq cg(n)$ for all $n > m$.
- $f \in \Theta(g)$ if $f \in \mathcal{O}(g)$ and $f \in \Omega(g)$. That is $\Theta = \mathcal{O} \cap \Omega$.

These are the definitions we expect you to know and use for this class. However, it is often useful to consider a slight alteration of these definitions:

- $f \in \mathcal{O}(g)$ if $\frac{f(n)}{g(n)}$ is bounded from above as $n$ increases (as $n \to \infty$).
- $f \in \Omega(g)$ if $\frac{f(n)}{g(n)}$ remains strictly positive as $n$ increases (as $n \to \infty$).
- $f \in \Theta(g)$ if $\frac{f(n)}{g(n)} \in (0, \infty)$ as $n$ increases (as $n \to \infty$).

These are equivalent when you notice that $c$ should be a real, finite, positive number.

| | $f(n)$ | $g(n)$ | $\Omega$ | $\mathcal{O}$ | $\Theta$ |
|---|---|---|---|---|---|
| (1) | $n^2 - 30n + 5$ | $0.7n^2 - 20n + 15$ | ★ | ★ | ★ |
| (2) | $100n^3 + 40n^2 - n$ | $0.5n^4 - 1000n^3$ | ○ | ★ | ○ |
| (3) | $5n^2 - n$ | $30n + 4$ | ★ | ○ | ○ |
| (4) | $n$ | $\sqrt{n}$ | ★ | ○ | ○ |
| (5) | $n^{\frac{3}{4}}$ | $\sqrt{n}$ | ★ | ○ | ○ |
| (6) | $\log_2 n$ | $\ln n$ | ★ | ★ | ★ |
| (7) | $n(\log_2 n)^5$ | $n\sqrt{n}$ | ○ | ★ | ○ |
| (8) | $2^n$ | $2^{2n}$ | ○ | ★ | ○ |
| (9) | $e^n$ | $2^n$ | ★ | ○ | ○ |
| (10) | $n!$ | $n^n$ | ○ | ★ | ○ |
| (11) | $n!$ | $2^n$ | ★ | ○ | ○ |
| (12) | $2^n$ | $2^{n+1}$ | ★ | ★ | ★ |

Below are some simple proofs of the above results.

(1) You may know to drop all but the highest order terms when comparing ratios of polynomials. This comes from L'Hôspital's rule:

$$\lim_{n\to\infty} \frac{n^2 - 30n + 5}{0.7n^2 - 20n + 15} = \lim_{n\to\infty} \frac{2n - 30}{1.4n - 20} = \lim_{n\to\infty} \frac{2}{1.4} = \frac{10}{7} \in (0, \infty).$$

We can also use this constant in place of $c$ in the formal definitions:

$$\begin{aligned}
\mathcal{O}: \quad n^2 - 30n + 5 \overset{?}{\leq} \frac{10}{7}(0.7n^2 - 20n + 15) \quad &\rightarrow \quad 7n^2 - 210n + 35 \overset{?}{\leq} 7n^2 - 200n + 150) \\
&\rightarrow \quad 0 - 10n - 115 \ < \ 0 \qquad \forall\, n > 0. \\
\Omega: \quad n^2 - 30n + 5 \overset{?}{\geq} \frac{1}{3}(0.7n^2 - 20n + 15) \quad &\rightarrow \quad 3n^2 - 90n + 15 \overset{?}{\geq} 0.7n^2 - 20n + 15) \\
&\rightarrow \quad 2.3n^2 - 70n - 0 \ > \ 0 \qquad \forall\, n > 31.
\end{aligned}$$

Thus $f$ is in both $\mathcal{O}$ and $\Omega$ of $g$.

(2) In this case, we get a limit of 0, so we only have $\mathcal{O}$. We can also see that

$$\begin{aligned}
\Omega: \quad 100n^3 + 40n^2 - n \overset{?}{\geq} c(0.5n^4 - 1000n^3) \quad &\rightarrow \quad 100n^2 + 40n^1 - 1 \overset{?}{\geq} \frac{c}{2}n^3 - 1000cn^2 \\
&\rightarrow \quad \left(2000 + \frac{200}{c}\right)n^2 + \frac{80}{c}n - \frac{2}{c} \overset{?}{\geq} n^3
\end{aligned}$$

In general, we find that the polynomial with the higher degree dominates.

(3) $f$ has higher degree, so we get $\Omega$.

(4) With fractional degree $\frac{n}{\sqrt{n}} = \sqrt{n}$ is unbounded, but positive (for positive $n$). Thus $\Omega$.

(5) Similar to above. The higher degree still dominates.

(6) Recall that $\log_2 n = \frac{\ln n}{\ln 2}$ so the ratio reduces to $\ln 2$, a positive constant. $\Theta$.

(7) After some simple algebra, we can compare $\log_2 n$ and $n^{\frac{1}{10}}$. The derivative of $\log_2 n$ is $\frac{1}{n \ln 2}$, so L'Hôspital's rule gives a limit of zero.

(8) Consider the ratio: $\frac{2^n}{2^2 n} = \frac{1}{2^n}$. This is clearly not bounded below (by a positive constant).

(9) Notice that $\mathsf{e}^n = (2 + 0.71\ldots)^n = 2^n + 2^{n-1}0.71\ldots$ which is clearly unbounded relative to $2^n$.

(10) The ratio

$$\frac{n!}{n^n} = \frac{n(n-1)(n-2)\ldots 1}{n^n} < \frac{n^{n-1}}{n^n} = \frac{1}{n}$$

is not bounded from below by a positive constant.

(11) If $n > 4$, then $n! = \prod_{i=1}^{n} i > \prod_{i=2}^{n} 2 = 2^n$. Thus $c = 1$ and $m = 4$ is sufficient to satisfy the definition of $\Omega$.

(12) $\frac{2^n}{2^{n+1}} = \frac{2^n}{2 \cdot 2^n} = \frac{1}{2}$ is positive and bounded. Hence $\Theta$.

**2. Properties of asymptotic notation** Let $f(n)$ and $g(n)$ be asymptotically positive functions (which means that $\lim_{n\to\infty} f(n) > 0$ and $\lim_{n\to\infty} g(n) > 0$). Prove or disprove each of the following conjectures

(1) $f(n) + g(n) \in \Theta(\min(f(n), g(n)))$.

(2) $f(n) + g(n) \in \Theta(\max(f(n), g(n)))$.

(3) $f(n) \in \mathcal{O}((f(n))^2)$.

(4) $f(n) \in \Omega((f(n))^2)$.

(5) $f(n) \in \mathcal{O}(g(n))$ implies $g(n) \in \Omega(f(n))$.

If the tree (and hence tree edges) represent a single path from the root to every other vertex, which edges constitute a second path?

(1) **false** — consider $f(n) = n$ and $g(n) = n^2$. $n^2 \notin \mathcal{O}(n)$.

(2) **true** — $\max\big(f(n), g(n)\big) \ \leq \ f(n) + g(n) \ \leq \ 2\max\big(f(n), g(n)\big)$

(3) **true** — $\frac{f(n)}{f(n)^2} = \frac{1}{f(n)}$. Since $f(n)$ is assumed to be asymptotically positive, this ratio is bounded from below by a positive constant.

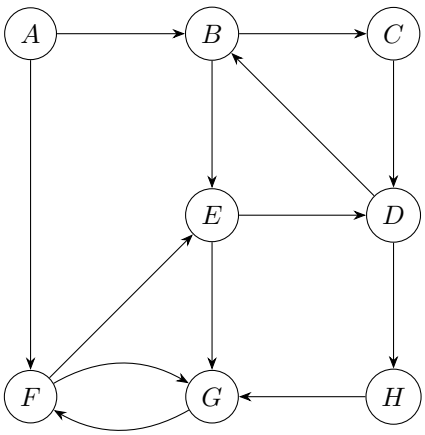
(4) **false** — consider $f(n) = n$.

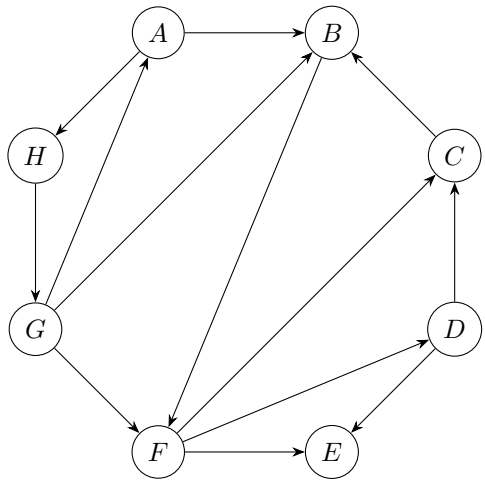
(5) **true** — $f(n) \leq cg(n)$ implies that $g(n) \geq \frac{1}{c} f(n)$.

**Interlude: Bridges.** Jamie will demo an algorithm for finding "bridges" in undirected graphs. See page 122 of https://iuuk.mff.cuni.cz/ koutecky/pruvodce-en-wip.pdf for a reference. Note that translation of this document into English is still a work in progress.

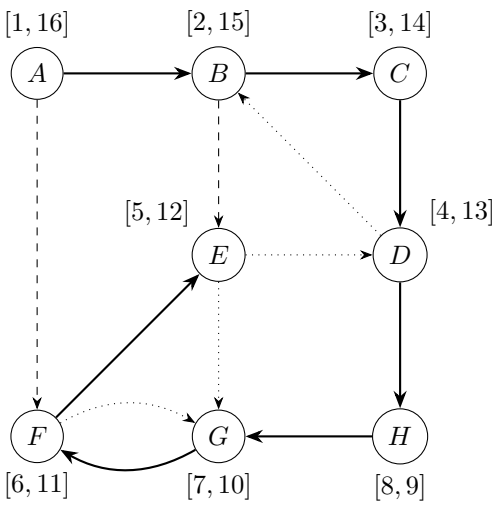
# Depth First Search

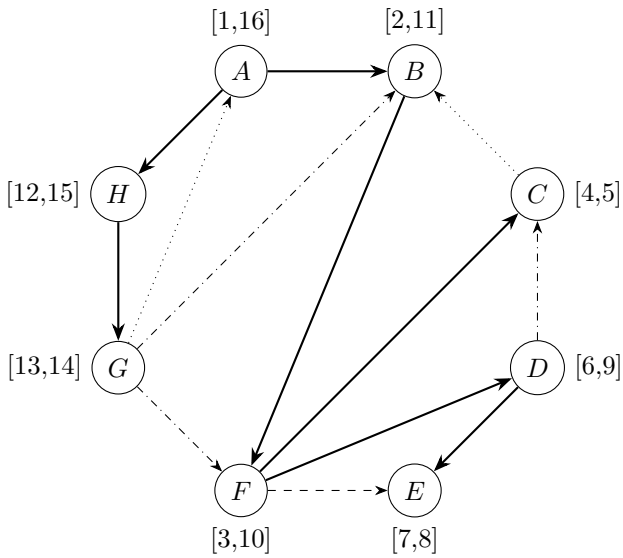
**3. DFS Runs** Perform depth-first search on each of the following graphs; whenever there's a choice of vertices, pick the one that is alphabetically first. Classify each edge as a tree, forward, back or cross edge.




The result can be seen in the following figures. Tree edges are thick, forward edges are dashed, back edges are dotted, and cross edges are dash dot. Each node is labeled with a pair of numbers, the first number is the "in" time (starting with 1), the second number is the "out" time.




**4. Singly Connected Graphs** A directed graph $G$ is *singly connected* if for every pair of vertices $u$ and $v$ there is at most one path from $u$ to $v$ in $G$. Give an efficient algorithm to determine whether or not a directed graph is singly connected.

If the tree (and hence tree edges) represent a single path from the root to every other vertex, which edges constitute a second path?

Both cross and forward edges imply additional paths to parts of the tree. Back edges imply cycles, but not paths. Simply flag when there is a cross or forward edge in DFS.

**Algorithm** `SinglyConnectedTF`$(G)$

```
for v ∈ Vertices do
    Run DFS rooted at v
    if a cross or forward edge is found then
        return false
    end if
end for
return true
```

Complexity: $\Theta(n(n+m))$ where $n = |V|$ and $m = |E|$.

**5. Longest Path In a DAG** Given a directed, acyclic graph (DAG) $G = (V, E)$ and two of its vertices $s$ and $t$, we want to compute the length of a longest path from $s$ to $t$.

**Proposition 2.1.** *Running DFS on an acyclic graph will never produce a back edge.*

Back edges necessarily imply a cycle.

Our goal will be to compute, for every node $u \in V$, the value the length $\ell(u)$ of the longest path from $s$ to $u$. The result is then $\ell(t)$.

Given a vertex $u$, let $\text{pred}(u)$ represent the in-neighborhood (or predecessors) of $u$. We have then, that

$$\ell(u) = 1 + \max_{v \in \text{pred}(u)} \ell(v).$$

If $\text{pred}(u) = \emptyset$ (so that there is no edge from $s$ to $u$), we will say $\ell(u) = -\infty$. In fact, we should initialize with $\ell(v) = -\infty$ for every vertex $v$.

From there, run DFS on $G$ (rooted at $s$ with $\ell(s) = 0$) and compute $\ell(v)$ in place as $\text{pred}(v)$ is explored. By Proposition 3.1, there can be no back edges, and we only need to traverse the tree once. If we memoize the following, we get linear time in the number of vertices.

**Algorithm** `LongestPathDAG`$(G, s, t)$

```
if s = t then
    return 0
else
    ℓ ← −∞
    for v ∈ pred(t) do
        k ← 1 + LongestPathDAG(G, s, v)
        if ℓ < k then
            ℓ ← k
        end if
    end for
end if
return ℓ
```

Note that it is possible to reverse this concept—to recursively compute the longest paths from $u$ to $t$—and get an equivalent algorithm.

**6. Count Paths In a DAG** Given a DAG $G = (V, E)$ and two of its vertices $s$ and $t$, design an algorithm that calculates the number of distinct paths from $s$ to $t$.

Our goal will be to compute, for every node $u \in V$, the number $c(u)$ of distinct paths from $u$ to $t$.

Given a vertex $u$, let $\text{succ}(u)$ represent the out-neighborhood (or successors) of $u$. We have then, that

$$c(u) = \sum_{v \in \text{succ}(u)} c(v).$$

If $\text{succ}(u) = \emptyset$ (and there is no path from u to $t$), then $c(u) = 0$.

For the moment, suppose that we have already run DFS rooted on $s$ and found a topological ordering of the vertices. Then, traversing the graph in reverse-topological order would allow us to use the above formula to compute $c(u)$ for every node $u$ very simply.

Ergo, we arrive at the following variant of DFS which, if properly memoized, runs in linear time in the number of vertices.

**Algorithm** `PathCountDAG`$(G, s, t)$

```
if s = t then
    return 1
else
    c ← 0
    for v ∈ succ(s) do
        c ← c + PathCountDAG(G, v, t)
    end for
end if
return c
```

Note that it is possible to reverse this concept—to recursively compute the number of paths from $s$ to $u$—and get an equivalent algorithm.

Jamie Fravel
jfravel@iuuk.mff.cuni.cz

# Asymptotics

**1. Comparing functions** Determine the asymptotic relationships between each pair of functions $f$ and $g$ in the following table (i.e. whether $f \in \mathcal{O}(g)$, $f \in \Omega(g)$ or $f = \Theta(g)$).

Lets start with a few different definitions of $\mathcal{O}$, $\Omega$ and $\Theta$.

Intuitively, $f \in \mathcal{O}(g)$ if $f$ grows at *most* as fast as $g$; $f \in \Omega(g)$ if $f$ grows at most as *least* as $g$ and $f \in \Theta(g)$ if $f$ grows *about* as fast as $g$.

Another intuition could be $f \in \mathcal{O}(g)$ if $g(n)$ never becomes irrelevantly small when compared to $f(n)$; $f \in \Omega(g)$ if $f(n)$ never becomes irrelevantly small when compared to $g(n)$; grows at *most* as fast as $g$ and $f \in \theta(g)$ if $f$ and $g$ never become irrelevant compared to each other.

Formally, $\mathcal{O}$, $\Omega$ and $\Theta$ are defined:

- $f \in \mathcal{O}(g)$ if there exist constants $c, m > 0$ such that $f(n) \leq cg(n)$ for all $n > m$.
- $f \in \Omega(g)$ if there exist constants $c, m > 0$ such that $f(n) \geq cg(n)$ for all $n > m$.
- $f \in \Theta(g)$ if $f \in \mathcal{O}(g)$ and $f \in \Omega(g)$. That is $\Theta = \mathcal{O} \cap \Omega$.

These are the definitions we expect you to know and use for this class. However, it is often useful to consider a slight alteration of these definitions:

- $f \in \mathcal{O}(g)$ if $\frac{f(n)}{g(n)}$ is bounded from above as $n$ increases (as $n \to \infty$).
- $f \in \Omega(g)$ if $\frac{f(n)}{g(n)}$ remains strictly positive as $n$ increases (as $n \to \infty$).
- $f \in \Theta(g)$ if $\frac{f(n)}{g(n)} \in (0, \infty)$ as $n$ increases (as $n \to \infty$).

These are equivalent when you notice that $c$ should be a real, finite, positive number.

| | $f(n)$ | $g(n)$ | $\Omega$ | $\mathcal{O}$ | $\Theta$ |
|---|---|---|---|---|---|
| (1) | $n^2 - 30n + 5$ | $0.7n^2 - 20n + 15$ | ★ | ★ | ★ |
| (2) | $100n^3 + 40n^2 - n$ | $0.5n^4 - 1000n^3$ | ○ | ★ | ○ |
| (3) | $5n^2 - n$ | $30n + 4$ | ★ | ○ | ○ |
| (4) | $n$ | $\sqrt{n}$ | ★ | ○ | ○ |
| (5) | $n^{\frac{3}{4}}$ | $\sqrt{n}$ | ★ | ○ | ○ |
| (6) | $\log_2 n$ | $\ln n$ | ★ | ★ | ★ |
| (7) | $n(\log_2 n)^5$ | $n\sqrt{n}$ | ○ | ★ | ○ |
| (8) | $2^n$ | $2^{2n}$ | ○ | ★ | ○ |
| (9) | $e^n$ | $2^n$ | ★ | ○ | ○ |
| (10) | $n!$ | $n^n$ | ○ | ★ | ○ |
| (11) | $n!$ | $2^n$ | ★ | ○ | ○ |
| (12) | $2^n$ | $2^{n+1}$ | ★ | ★ | ★ |

Below are some simple proofs of the above results.

(1) You may know to drop all but the highest order terms when comparing ratios of polynomials. This comes from L'Hôspital's rule:

$$\lim_{n\to\infty} \frac{n^2 - 30n + 5}{0.7n^2 - 20n + 15} = \lim_{n\to\infty} \frac{2n - 30}{1.4n - 20} = \lim_{n\to\infty} \frac{2}{1.4} = \frac{10}{7} \in (0, \infty).$$

We can also use this constant in place of $c$ in the formal definitions:

$$\begin{aligned}
\mathcal{O}: \quad n^2 - 30n + 5 \overset{?}{\leq} \frac{10}{7}(0.7n^2 - 20n + 15) \quad &\to \quad 7n^2 - 210n + 35 \overset{?}{\leq} 7n^2 - 200n + 150) \\
&\to \quad 0 - 10n - 115 \ < \ 0 \qquad \forall\, n > 0. \\
\Omega: \quad n^2 - 30n + 5 \overset{?}{\geq} \frac{1}{3}(0.7n^2 - 20n + 15) \quad &\to \quad 3n^2 - 90n + 15 \overset{?}{\geq} 0.7n^2 - 20n + 15) \\
&\to \quad 2.3n^2 - 70n - 0 \ > \ 0 \qquad \forall\, n > 31.
\end{aligned}$$

Thus $f$ is in both $\mathcal{O}$ and $\Omega$ of $g$.

(2) In this case, we get a limit of 0, so we only have $\mathcal{O}$. We can also see that

$$\begin{aligned}
\Omega: \quad 100n^3 + 40n^2 - n \overset{?}{\geq} c(0.5n^4 - 1000n^3) \quad &\to \quad 100n^2 + 40n^1 - 1 \overset{?}{\geq} \frac{c}{2}n^3 - 1000cn^2 \\
&\to \quad \left(2000 + \frac{200}{c}\right)n^2 + \frac{80}{c}n - \frac{2}{c} \overset{?}{\geq} n^3
\end{aligned}$$

In general, we find that the polynomial with the higher degree dominates.

(3) $f$ has higher degree, so we get $\Omega$.

(4) With fractional degree $\frac{n}{\sqrt{n}} = \sqrt{n}$ is unbounded, but positive (for positive $n$). Thus $\Omega$.

(5) Similar to above. The higher degree still dominates.

(6) Recall that $\log_2 n = \frac{\ln n}{\ln 2}$ so the ratio reduces to $\ln 2$, a positive constant. $\Theta$.

(7) After some simple algebra, we can compare $\log_2 n$ and $n^{\frac{1}{10}}$. The derivative of $\log_2 n$ is $\frac{1}{n \ln 2}$, so L'Hôspital's rule gives a limit of zero.

(8) Consider the ratio: $\frac{2^n}{2^2 n} = \frac{1}{2^n}$. This is clearly not bounded below (by a positive constant).

(9) Notice that $\mathsf{e}^n = (2 + 0.71\ldots)^n = 2^n + 2^{n-1}0.71\ldots$ which is clearly unbounded relative to $2^n$.

(10) The ratio

$$\frac{n!}{n^n} = \frac{n(n-1)(n-2)\ldots 1}{n^n} < \frac{n^{n-1}}{n^n} = \frac{1}{n}$$

is not bounded from below by a positive constant.

(11) If $n > 4$, then $n! = \prod_{i=1}^{n} i > \prod_{i=2}^{n} 2 = 2^n$. Thus $c = 1$ and $m = 4$ is sufficient to satisfy the definition of $\Omega$.

(12) $\frac{2^n}{2^{n+1}} = \frac{2^n}{2 \cdot 2^n} = \frac{1}{2}$ is positive and bounded. Hence $\Theta$.

**2. Properties of asymptotic notation** Let $f(n)$ and $g(n)$ be asymptotically positive functions (which means that $\lim_{n\to\infty} f(n) > 0$ and $\lim_{n\to\infty} g(n) > 0$). Prove or disprove each of the following conjectures

(1) $f(n) + g(n) \in \Theta(\min(f(n), g(n)))$.

(2) $f(n) + g(n) \in \Theta(\max(f(n), g(n)))$.

(3) $f(n) \in \mathcal{O}((f(n))^2)$.

(4) $f(n) \in \Omega((f(n))^2)$.

(5) $f(n) \in \mathcal{O}(g(n))$ implies $g(n) \in \Omega(f(n))$.

If the tree (and hence tree edges) represent a single path from the root to every other vertex, which edges constitute a second path?

(1) **false** — consider $f(n) = n$ and $g(n) = n^2$. $n^2 \notin \mathcal{O}(n)$.

(2) **true** — $\max\big(f(n), g(n)\big) \ \leq \ f(n) + g(n) \ \leq \ 2\max\big(f(n), g(n)\big)$

(3) **true** — $\frac{f(n)}{f(n)^2} = \frac{1}{f(n)}$. Since $f(n)$ is assumed to be asymptotically positive, this ratio is bounded from below by a positive constant.

(4) **false** — consider $f(n) = n$.

(5) **true** — $f(n) \leq cg(n)$ implies that $g(n) \geq \frac{1}{c} f(n)$.

**Interlude: Bridges.** Jamie will use SYGA to demo an algorithm for finding "bridges" in undirected graphs. See page 122 of https://iuuk.mff.cuni.cz/ koutecky/pruvodce-en-wip.pdf for a reference. Note that translation of this document into English is still a work in progress.
This also serves as our introduction to the SYGA (See Your Graph Algorithm) visualizer.

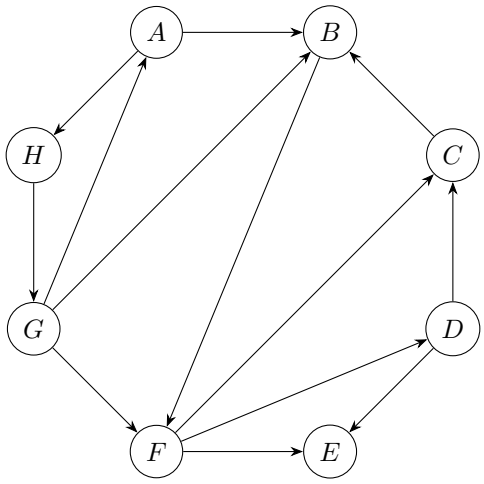
## Depth First Search

**3. DFS Runs — SYGA** Starting at vertex $A$, perform depth-first search on the following graph in SYGA:

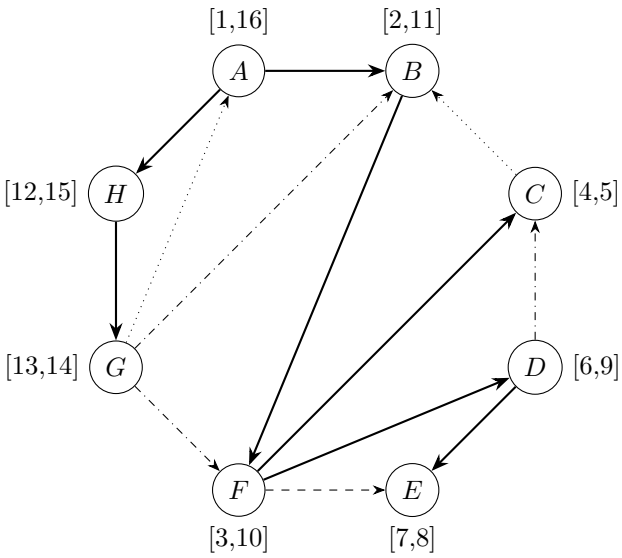
syga.space/exercises/week-2/practice-depth-first-search.

Whenever there's a choice of vertices, proceed in alphabetical order. First, find the tree edges, then classify the non-tree edges into back, forward and cross edges.

Finally you can compare your solution to one generated algorithmically. Feel free to change the graph (at the top of the script) or explore the algorithm.




The result can be seen in the following figures. Tree edges are thick, forward edges are dashed, back edges are dotted, and cross edges are dash dot. Each node is labeled with a pair of numbers, the first number is the "in" time (starting with 1), the second number is the "out" time.




**4. Singly Connected Graphs — SYGA** A directed graph $G$ is *singly connected* if for every pair of vertices $u$ and $v$ there is at most one path from $u$ to $v$ in $G$. Follow this link to the SYGA exercise:

syga.space/exercise/week-2/singly-connected-graphs

Adjust the algorithm to determine whether or not a directed graph is singly connected. Your finished algorithm should set `flag = 1` whenever you find a counterexample.

There are three graphs at the top of the script for you to try your solution on. The first two are *not* singly connected.

Tree edges represent one path from the root to every other (reachable) node in the graph. What kinds of non-tree edges are there?

Both cross and forward edges imply additional paths to parts of the tree. Back edges imply cycles, but not paths. Simply set `flag = 1` when there is a cross or forward edge in DFS. In the second graph, DFS rooted at "A" will never reach "C", will miss a forward edge and erroneously return "true". To get a true result, the process must reach *every*. In particular, running from "C" gives the correct result of "false".

**Algorithm** `SinglyConnectedTF`$(G)$

```
for v ∈ Vertices do
    Run DFS rooted at v
    if a cross or forward edge is found then
        return false
    end if
end for
return true
```

Complexity: $\Theta(n(n+m))$ where $n = |V|$ and $m = |E|$.

**5. Longest Path In a DAG** Given a directed, acyclic graph (DAG) $G = (V, E)$ and two of its vertices $s$ and $t$, we want to compute the length of a longest path from $s$ to $t$.

**Proposition 3.1.** *Running DFS on an acyclic graph will never produce a back edge.*

Back edges necessarily imply a cycle.

Our goal will be to compute, for every node $u \in V$, the value the length $\ell(u)$ of the longest path from $s$ to $u$. The result is then $\ell(t)$.

Given a vertex $u$, let $\text{pred}(u)$ represent the in-neighborhood (or predecessors) of $u$. We have then, that

$$\ell(u) = 1 + \max_{v \in \text{pred}(u)} \ell(v).$$

If $\text{pred}(u) = \emptyset$ (so that there is no edge from $s$ to $u$), we will say $\ell(u) = -\infty$. In fact, we should initialize with $\ell(v) = -\infty$ for every vertex $v$.

From there, run DFS on $G$ (rooted at $s$ with $\ell(s) = 0$) and compute $\ell(v)$ in place as $\text{pred}(v)$ is explored. By Proposition 3.1, there can be no back edges, and we only need to traverse the tree once. If we memoize the following, we get linear time in the number of vertices.

**Algorithm** `LongestPathDAG`$(G, s, t)$

```
if s = t then
    return 0
else
    ℓ ← −∞
    for v ∈ pred(t) do
        k ← 1 + LongestPathDAG(G, s, v)
        if ℓ < k then
            ℓ ← k
        end if
    end for
end if
return ℓ
```

Note that it is possible to reverse this concept—to recursively compute the longest paths from $u$ to $t$—and get an equivalent algorithm.

**6. Count Paths In a DAG** Given a DAG $G = (V, E)$ and two of its vertices $s$ and $t$, design an algorithm that calculates the number of distinct paths from $s$ to $t$.

Our goal will be to compute, for every node $u \in V$, the number $c(u)$ of distinct paths from $u$ to $t$.

Given a vertex $u$, let $\text{succ}(u)$ represent the out-neighborhood (or successors) of $u$. We have then, that

$$c(u) = \sum_{v \in \text{succ}(u)} c(v).$$

If $\text{succ}(u) = \emptyset$ (and there is no path from u to $t$), then $c(u) = 0$.

For the moment, suppose that we have already run DFS rooted on $s$ and found a topological ordering of the vertices. Then, traversing the graph in reverse-topological order would allow us to use the above formula to compute $c(u)$ for every node $u$ very simply.

Ergo, we arrive at the following variant of DFS which, if properly memoized, runs in linear time in the number of vertices.

```
Algorithm PathCountDAG(G, s, t)
  if s = t then
      return 1
  else
      c ← 0
      for v ∈ succ(s) do
          c ← c + PathCountDAG(G, v, t)
      end for
  end if
  return c
```

Note that it is possible to reverse this concept—to recursively compute the number of paths from $s$ to $u$—and get an equivalent algorithm.

Jamie Fravel
jfravel@iuuk.mff.cuni.cz

# Topological Orders

**1. Peeling a Graph** You're given a connected undirected graph $G$. What is the fastest way to determine an order of the vertices $v_1, \ldots, v_n$ such that for each $i$, after we have removed $v_1, \ldots, v_i$, the remaining graph is still connected?

If $G$ was a DAG, you could just remove nodes in reverse-topological order (e.g., $v_n, \ldots, v_1$) because there necessarily exists no edge from $v_n$ to any other node still in the graph. How is this order found?

Being a non-DAG, $G$ doesn't have a well-defined topological ordering, but we can take some inspiration from the ordering process.

In this undirected case, DFS produces a tree with some back edges. Removing a back edge will never disconnect the graph, so they can be ignored. You can also remove nodes with only one tree edge (leaves). After each of the successors of a node $u$ have been removed, then $u$ is a leaf and can also be removed. this evokes DFS.

Start DFS in any node of $G$. When a node is left (after removing its whole sub-tree), remove it. The order is thus given by decreasing "in" time (or increasing "out" time).

**2. Parallel Planning** When building a house or starting a business, you might construct a *dependency graph*. This is a DAG $G = (V, E)$ with vertices representing tasks and edges representing dependencies between those tasks. In particular, if $(u, v) \in E$, then task $u$ must be completed before $v$ can be started. The nodes will also be weighted by a function $\ell : V \to \mathbb{N}$ where $\ell(u)$ is the length of time that task $u$ will require to complete.

(a) You are given a target date $T \in \mathbb{N}$ and you need to compute, for each task $u \in V$, the *latest* time $t(u)$ at which you can start executing $u$ so that the entire project is finished by time $T$. Assume that you are able to perform arbitrarily many tasks in parallel so long as none of them depend on each other.

(b) A vertex $u$ is called *critical* if any delays in completing task $u$ (an increase in $\ell(u)$) necessarily delays the end time of the whole project. How would you detect all critical vertices in a given dependency graph?

(a) Any sink tasks $u$ (those without any successors) may be scheduled to conclude at time $T$ by setting $t(u) = T - \ell(u)$. Otherwise, it must be scheduled early enough that all of its successors conclude before $T$. Recursively, we get

$$t(u) = \begin{cases} T - l(u) & \text{if } \mathrm{adj}(u) = \emptyset, \\ \min\limits_{v \in \mathrm{adj}(u)} t(v) - l(u) & \text{otherwise.} \end{cases}$$

Given a topological ordering of the vertices ($V = \{v_1, \ldots, v_n\}$ such that $(v_i, v_j) \in E$ implies $i < j$), $t(v_i)$ can be computed by sequentially considering the indices $i = n, \ldots, 1$.

(b) Suppose that $G$ has several source nodes $s_1, \ldots, s_k$ and consider new node $\bar{s}$ along with the edges $(\bar{s}, s_i)$ for each $i \in (1, \ldots, k)$. Now $G$ has only one source and, by setting $\ell(\bar{s}) = 0$, we haven't changed the space of solutions. The same can be done for sink nodes. We may therefore assume, without loss of generality, that $G$ has a single source $s$ and a single sink $t$.

In this case, we define the length of a path $P$ to be the sum of the durations of each node in $P$. Let $d(u)$ represent the length of a longest path from $s$ to $u$ in $G$ so that $\tau = d(t)$ is the minimum deadline $T$ under which all tasks can be completed. A node $u$ is critical if increasing $\ell(u)$ would also increase $\tau$.

Let us now denote by $f(u)$ the length of a longest path from $u$ to $t$. Then a node $u$ is critical if $d(u) + f(u) = \tau$ which occurs if and only if $u$ lies on some longest path from $s$ to $t$. The values of $d(u)$ and $f(u)$ can be computed in linear time following the topological order of $G$.

# Connectedness

**3. Computopia** The police department in the city of Computopia has made all streets one-way. The mayor contends that there is still a way to legally drive from any intersection in the city to any other intersection, but the opposition is not convinced. A computer program is needed to determine whether the mayor is right. However, the city elections are coming up soon, and there is just enough time to run a linear-time algorithm.

- Formulate this problem graph-theoretically, and explain why it can indeed be solved in linear time.
- Suppose it now turns out that the mayor's original claim is false. She next claims something weaker: If you start driving from the town hall, navigating one-way streets, then no matter where you arrive, there is always a way to legally drive back to the town hall. Formulate this weaker property as a graph-theoretic problem and carefully show how it can also be checked in linear time.

- Define a directed graph $G$ with nodes being the intersections and edges the streets. Then we want to know if $G$ is strongly connected.

  Now, let the node $s$ represent city hall. We first check, by DFS, whether all nodes can be reached from $s$. Then, we reverse the directions of all streets and repeat DFS from $s$ to check if $s$ can be reached from all other nodes in $G$. $G$ is strongly connected if and only if *both* DFS runs reach all nodes.
- We first identify the set $U$ of nodes that can be reached from $s$ by running a DFS starting in $s$. Then we reverse the graph and check if we can again reach all of the nodes in $U$ from $s$

  In terms of SCCs, we want to check that there is no street leaving the SCC containing $s$.

**4. Semi-connectedness.** We say that a directed graph $G = (V, E)$ is *semi-connected* if for every pair of vertices $u, v \in V$ there is a directed path from $u$ to $v$ or from $v$ to $u$ (or both). Describe a linear-time algorithm which determines whether a given directed graph is semi-connected. *Hint: Consider first the case when $G$ is acyclic.*

First, consider a DAG $G$ with topologically ordered vertices $v_1, \ldots, v_n$. If there exists a pair of consecutive nodes $v_i, v_{i+1}$ such that $(v_i, v_{i+1}) \notin E$, then there is no path from $v_i$ to $v_{i+1}$ and $G$ is *not* semi-connected. Otherwise, there must clearly be a path from every vertex to each of those which come later in the order. Thus we only need to verify the edge $(v_i, v_{i+1})$ for each $i = 1, \ldots, n-1$. A linear number of checks.

Now, if $G$ is cyclic, we find the strongly connected components of $G$ before constructing an acyclic, auxiliary graph $G' = (V', E')$. The algorithm for detecting SCCs gives the components $C_1, \ldots, C_t$ in topological order, so let the node $c_i$ represent the SCC $C_i$ and include the edge $(c_i, c_j)$ if and only if there is an edge from $C_i$ to $C_j$. Recall that, if $G'$ contained a cycle, then each of the distinct SCCs in said cycle are actually one larger SCC. Thus, $G'$ is a DAG and we return to the acyclic case.

Jamie Fravel
jfravel@iuuk.mff.cuni.cz

# Strongly connected components

**1. Transitive Reduction** The *transitive closure* $G^T$ of a directed graph $G$ is a directed graph with the same set of vertices as $G$, that contains any edge $(u, v)$ if and only if there is a directed path from $u$ to $v$ in $G$. A *transitive reduction* of $G$ is a graph with the smallest possible number of edges whose transitive closure is $G^T$. The same graph may have several transitive reductions.

(a) Describe an efficient algorithm to compute the transitive closure of a given directed graph.

(b) Prove that if $G$ is an acyclic graph, it has a unique transitive reduction.

(c) Describe an efficient algorithm to compute a transitive reduction of a given directed graph.

(a) One way is to run DFS or BFS from every node $u \in V$ to find nodes $v$ which are reachable from $u$ by a path. This gives the edges of the transitive closure.

(b) If $(u, v) \in E$ is such that $G' = (V, E \setminus \{(u, v)\})$ contains a path from $u$ to $v$, then $G'$ has the same transtive closure as $G$ and $(u, v)$ can be removed from $G$. Repeat until no edge can be removed. No matter how we choose the order of edges we always arrive at the same DAG.

(c) Construct the SCC component graph. Replace each SCC with a cycle, find the transitive reduction of the acyclic component graph. For each node $u$, we store the list of reachable nodes. An $(u, v)$ can be removed, if $v \in R(w)$ for some other neighbor $w$ of $u$. Set $R(u)$ is the union of $R(v)$ over all neighbors $v$ of $u$, plus $u$. Use topological order for effectiveness.

# BFS and Shortest Paths

**2. Bipartite Graph Check** Describe an algorithm that checks if a given undirected graph $G = (V, E)$ is bipartite. The algorithm should have complexity $O(|V| + |E|)$.

**Proposition 5.1.** *An undirected graph is bipartite if and only if it contains no cycles with an odd number of edges.*

What class of edges imply a cycle?

Run BFS with computing the distance $d(s)$ from a source node $s$ (that can be set arbitrarily for each connected component). The graph is bipartite if there is no edge $\{u, v\}$ in which $d(u)$ is odd and $d(v)$ even or vice versa (the edge is undirected). We only ever need to check non-tree edges (back edges) as $d(u)$ is assigned by tree edges.

This can also be done with DFS or by assigning the nodes to a partition, alternating after each tree edge.

**3. Broken Taxi** A taxi broke down in Manhattan, so it can only drive straight or turn right. Manhattan is an $n \times n$ grid, but some of the roads may be missing or blocked. So some of the edges may be missing in the map. You are given the location of the taxi, the location of a repair shop, and your task is to give instructions to the driver on how to reach the shop and burn as little gas as possible.

Construct a directed graph in which each intersection $u$ is represented with four nodes, each corresponding to one direction the car can be oriented in $(u^{\uparrow}, u^{\downarrow}, u^{\leftarrow}, u^{\rightarrow})$. These state nodes represent a situation in which the car is entering the intersection and is currently oriented in the direction of the arrow. We then define the edges as follows. Consider state node $u^{\uparrow}$. If $v$ is above $u$ and $\{u, v\}$ is an edge of the map, then we add edge $(u^{\uparrow}, v^{\uparrow})$ to the graph. If $v$ is to the right of $u$ and $\{u, v\}$ is an edge, then we add edge $(u^{\uparrow}, v^{\rightarrow})$ representing the fact that the car turned right. We proceed similarly with all the other directions (the situation is only rotated). We will assume that the car starts at an intersection (the closest one that is in front of the car) and the repair shop is also at some crossroad. Then we simply run BFS on the graph constructed in this way to find a shortest path.

**4. #Shortest Paths** Given a directed graph $G = (V, E)$ and its two vertices $s, t$, design an algorithm computing the number of shortest paths from $s$ to $t$.

Run BFS on $G$ starting in node $s$. For each node $v \in V$, we compute the distance $d(v)$ from $s$ as usual, but in addition, we also compute $c(v)$ as the number of paths of length $d(v)$ found. When considering a node $u$ and its neighbor $v$, we use the following rule to update $d(v)$ and $c(v)$ based on $d(u)$ and $c(u)$.

1. If the value of $d(v)$ hasn't been set yet, then we set $d(v) = d(u) + 1$ and $c(v) = c(u)$.
2. If $d(u) + 1 = d(v)$, then we leave $d(v)$ unchanged and we set $c(v) = c(v) + c(u)$.
3. If $d(u) + 1 > d(v)$, we do nothing.

**5. Pouring Water** We have three containers whose sizes are 10 pints, 7 pints, and 4 pints, respectively. The 7-pint and 4-pint containers start out full of water, but the 10-pint container is initially empty. We are allowed one type of operation: pouring the contents of one container into another, stopping only when the source container is empty or the destination container is full. We want to know if there is a sequence of pourings that leaves exactly 2 pints in the 7- or 4-pint container.

(a) Model this as a graph problem: give a precise definition of the graph involved and state the specific question about this graph that needs to be answered.

(b) What algorithm should be applied to solve the problem?

(c) Find the answer by applying the algorithm.

(a) A state of the task is defined as a triple $(a, b, c)$ where $a \in \{0, \dots 10\}$ is the amount of water in the 10-pint container, $b \in \{0, \dots 7\}$ is the amount of water in the 7-pint container, and $c \in \{0, \dots 4\}$ is the amount of water in the 4-pint container. In addition, we start with the 7-pint and 4-pint container full and we neither spill any water, nor we gain any more water. So we always have $a + b + c = 11$.

Pouring water from one container to another changes the state. Consider for instance state $(3, 5, 3)$. Pouring the water from the 10-pint container to the 4-pint container changes the state to $(2, 5, 4)$. If we then pour the water from the 7-pint container to the 10-pint container, the state is further changed to $(7, 0, 4)$.

We can thus define a directed graph $G = (V, E)$ with the nodes $V$ equal to the set of states. We have $(u, v) \in E$ for two states $u$ and $v$ if a water pouring action can change $u$ into $v$.

The number of the nodes $|V|$ can be trivially upper bounded by $11 \cdot 8 \cdot 5 = 440$. Since all states $(a, b, c)$ must satisfy $a + b + c = 11$, the state is actually specified only by a pair of $b$ and $c$, because then $a = 11 - b - c$. Thus$|V| \leq 40$. We can further observe that it is not possible to have $b = c = 0$, because $a \leq 10 < 11$. Thus $|V| \leq 39$.

The number of edges can be estimated as $|E| \leq 6|V| = 234$, because each state has output degree at most 6 (only 6 different actions are available).

Denote $s = (0, 7, 4)$ the initial state. We are then looking for a path from $s$ to one of the states $(a, b, c)$ where $b = 2$ or $c = 2$.

(b) We can use DFS or BFS on the graph, in the latter case we get a shortest sequence of pouring actions as well. Note that we do not need to construct the graph beforehand, it is enough to generate the neighbour states for each visited states.

(c) Say by DFS we can find a sequence $(0, 7, 4) \to (4, 7, 0) \to (10, 1, 0) \to (6, 1, 4) \to (6, 5, 0) \to (2, 5, 4) \to (2, 7, 2)$. The last state in the sequence has 2 pints in the 4-pint container.

**6. Ghost** There is a maze and at position $S$ a ghost. Think of the maze as a grid where each cell is either a hallway or a wall. The ghost can walk through walls, but it moves through walls 4 times slower than through hallways. The ghost wants to reach a position $T$. Find the shortest path.

Construct a graph based on the grid where each hallway is represented with a single edge and each wall is represented with four edges, then run BFS on this graph.

**7. Guard** We have a maze on an $n \times n$ grid with walls, a hero, and a treasure. We want to find a way for the hero to reach the treasure as quickly as possible, but not run into a guard (that is, appear in the same cell as the guard at the same time). The guard has a fixed route of length $L$ as a sequence of adjacent cells and it goes back and forth on this route. How to find such a shortest route as quickly as possible? What is the complexity of your algorithm in terms of $N$ and $L$?

Construct the graph $G = (V, E)$ based on the maze. Then make$2L$ copies of each node. In particular, each node $v$ is replaced with nodes $(v, t)$ where $t$ represents time modulo $2L$. We For each edge $(u, v)$ in the original graphand each $t$, we add an edge $((u, t), (u, t + 1))$ unless the guard is on vertex $v$ at time $t + 1$. In addition, we add edges $((u, t), (u, t + 1))$ that allow to wait on a position till the guard moves away. Then, we run BFS on this graph.